

Kubernetes Introduction

1. Why Kubernetes Exists

Suppose we have a simple Spring Boot application:

```
User
  ↓
Spring Boot App
  ↓
MySQL
```

We can containerize the application using Docker:

```
docker build -t myapp .
docker run -p 8080:8080 myapp
```

For a small application, this works perfectly.

But an important question appears when the application grows:

| If Docker can already run containers, why do we need Kubernetes?

2. What Problem Does Docker Solve?

Docker helps us package an application together with everything it needs:

```
Application
Libraries
Runtime
Dependencies
Configuration
```

This package becomes a:

```
Docker Image
```

From the image, we create:

```
Container
```

So Docker primarily solves:

| How do I package and run my application consistently?

But Docker alone does not automatically solve large-scale container management.

For example:

- How do we manage hundreds of containers?
- What happens when a container crashes?
- What happens when an entire server crashes?
- How do we scale applications?
- How do we distribute containers across multiple machines?
- How do we update applications without downtime?
- How do services find each other?
- How do we expose applications?
- How do we manage configuration?
- How do we manage CPU and memory resources?

This is where Kubernetes becomes useful.

3. Think at Production Scale

Imagine a platform grows from:

```
1 Server  
1 Application  
1 Container
```

to something like:

```
100 Servers
500 Application Containers
50 Databases / Caches / Message Brokers
Multiple Microservices
```

Managing containers manually is no longer practical.

We need a system that can continuously manage the infrastructure for us.

That system is Kubernetes.

4. Problem 1 — What If a Container Crashes?

Imagine a server running three containers:

```
Server A

Container 1 
Container 2 
Container 3 
```

Without orchestration, someone might have to manually detect the failure:

```
Engineer notices failure
    ↓
SSH into server
    ↓
docker run ...
```

This approach cannot scale.

Instead, what we actually want is to tell a system:

```
I always want 3 copies of my application running.
```

The system should continuously compare:

```
Desired State = 3
```

```
Actual State = 2
```

and automatically correct the situation:

```
Actual State → 3
```

This is one of the most important ideas behind Kubernetes.

5. Desired State vs Actual State

Kubernetes works around the concept of **desired state**.

Instead of manually controlling every container, we describe what we want.

For example:

```
I want:
```

```
3 copies of application X  
using image version 2  
with 1 GB memory  
exposed on port 8080
```

This becomes the:

```
Desired State
```

Kubernetes then observes what currently exists:

```
Actual State
```

and continuously tries to make:

```
Actual State = Desired State
```

This continuous correction process is called:

Reconciliation

This idea appears throughout Kubernetes.

6. Problem 2 — Scaling

Suppose an application initially needs only one container:

```
Application
  ↓
Container
```

Traffic increases and one container is no longer enough.

Now we might need:

```
Container
Container
Container
Container
Container
```

Without Kubernetes, we might manually start multiple containers.

With Kubernetes, we simply declare:

```
replicas = 5
```

Kubernetes takes responsibility for maintaining five running copies.

This is called **declarative management**.

Instead of describing every step Kubernetes should perform, we describe the final state we want.

7. Problem 3 — Server Failure

Container failure is one problem.

Server failure is even more serious.

Suppose:

```
Server A
├─ Container 1
├─ Container 2
└─ Container 3
```

If Server A fails, all three containers disappear.

In a Kubernetes cluster with multiple machines:

```
Node A ✖
Node B ✔
Node C ✔
```

Kubernetes can recreate the required workloads on the remaining healthy machines.

For example:

```
Before Failure

Node A
├─ App1
└─ App2

Node B
└─ App3
```

Node A fails.

Kubernetes can eventually recreate the missing applications:

```
Node B
├─ App3
└─ App1

Node C
└─ App2
```

This ability is part of Kubernetes' **self-healing** behavior.

8. Problem 4 — Deploying a New Version

Suppose ten instances of an application are running:

```
myapp:v1
```

Now we want to deploy:

```
myapp:v2
```

A naive deployment would be:

```
Stop all v1 containers
    ↓
Start all v2 containers
```

For some period, we could have:

```
0 Containers Running
```

which means:

```
Downtime
```

Kubernetes can instead perform a **rolling update**.

```
v1 v1 v1 v1
  ↓
v2 v1 v1 v1
  ↓
v2 v2 v1 v1
  ↓
v2 v2 v2 v1
  ↓
v2 v2 v2 v2
```

Old instances are gradually replaced with new ones while the application remains available.

9. Problem 5 — Where Should a Container Run?

Imagine a cluster containing several machines:

```
Node A
CPU Usage: 90%

Node B
CPU Usage: 20%

Node C
CPU Usage: 50%

Node D
CPU Usage: 70%

Node E
CPU Usage: 10%
```

A new application requires:

```
2 CPU
4 GB RAM
```

Someone needs to decide which machine should run it.

Developers should not manually inspect every machine and choose one.

Kubernetes automatically makes this placement decision through a component called the:

```
kube-scheduler
```

10. What Is Kubernetes?

A useful definition is:

Kubernetes is a container orchestration platform that manages the deployment, scaling, networking, availability, and lifecycle of containerized applications across a cluster of machines.

The simplest mental model is:

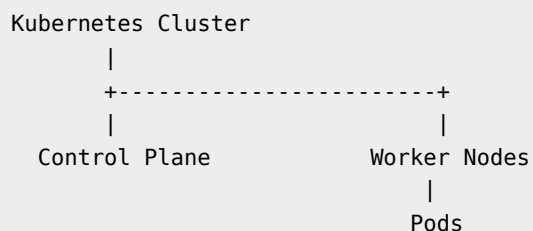
```
Docker
=
Runs Containers

Kubernetes
=
Manages Containers at Scale
```

Docker and Kubernetes therefore solve related but different problems.

11. Basic Kubernetes Mental Model

Before understanding Kubernetes architecture, remember this structure:



|
Containers

There are two broad responsibilities:

Control Plane

Manages the cluster and makes decisions.

Worker Nodes

Machines where our actual applications run.

12. From One Machine to a Cluster

Imagine one server:

```
Server
├─ Container A
├─ Container B
└─ Container C
```

For example:

```
Container A → User Service
Container B → Payment Service
Container C → Order Service
```

Managing three containers manually is possible.

Now imagine:

```
Server 1
Server 2
Server 3
...
Server 100
```

with hundreds of containers.

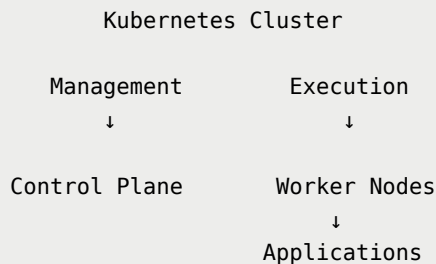
We now need answers to questions such as:

```
Which container should run on which server?  
  
What if one server becomes full?  
  
What if a container crashes?  
  
What if an entire server crashes?  
  
How many copies of an application should exist?  
  
Where should a newly deployed application run?
```

This creates the need for a centralized management system.

13. Separating Management from Execution

Kubernetes separates machines into two broad roles.



The **Control Plane** manages the cluster.

The **Worker Nodes** run application workloads.

A simple analogy is:

```
Control Plane = Brain  
Worker Nodes = Workers
```

14. What Is a Node?

A **Node** is simply a machine participating in a Kubernetes cluster.

It may be:

- A physical server
- A virtual machine
- A cloud VM

For example:

```
AWS EC2 Virtual Machine
      ↓
Kubernetes Node
```

A Kubernetes cluster contains multiple nodes:

```
Kubernetes Cluster
├─ Node 1
├─ Node 2
├─ Node 3
└─ Node 4
```

Some nodes may perform control-plane responsibilities.

Other nodes act as workers.

15. Giving Kubernetes Its First Job

Suppose our application has already been packaged into a container image:

```
coderarmy/myapp:v1
```

Now we want Kubernetes to run it.

We need some way to communicate with the Kubernetes cluster.

This introduces an important tool:

kubectl

16. What Is **kubectl** ?

kubectl is the command-line tool used to communicate with Kubernetes.

For example:

```
kubectl get nodes
```

Its simple English meaning is:

| Kubernetes, show me the nodes in this cluster.

For now, the important idea is not the command syntax.

Remember:

```
User  
↓  
kubectl  
↓  
Kubernetes
```

kubectl is normally installed on your laptop or another management machine.

17. API Server

When we use **kubectl**, something inside Kubernetes must receive our request.

That component is:

```
kube-apiserver
```

or simply:

| **API Server**

Think of the API Server as the **gateway or front desk of the Kubernetes Control Plane**.

The communication looks like:

```
kubectl
  ↓
API Server
```

This is an important relationship to remember:

```
kubectl → API Server
```

Why Is It Called an API Server?

Kubernetes exposes an API.

Through it, we can request operations such as:

```
Create something
Delete something
Show something
Update something
```

For example:

```
kubectl get nodes
```

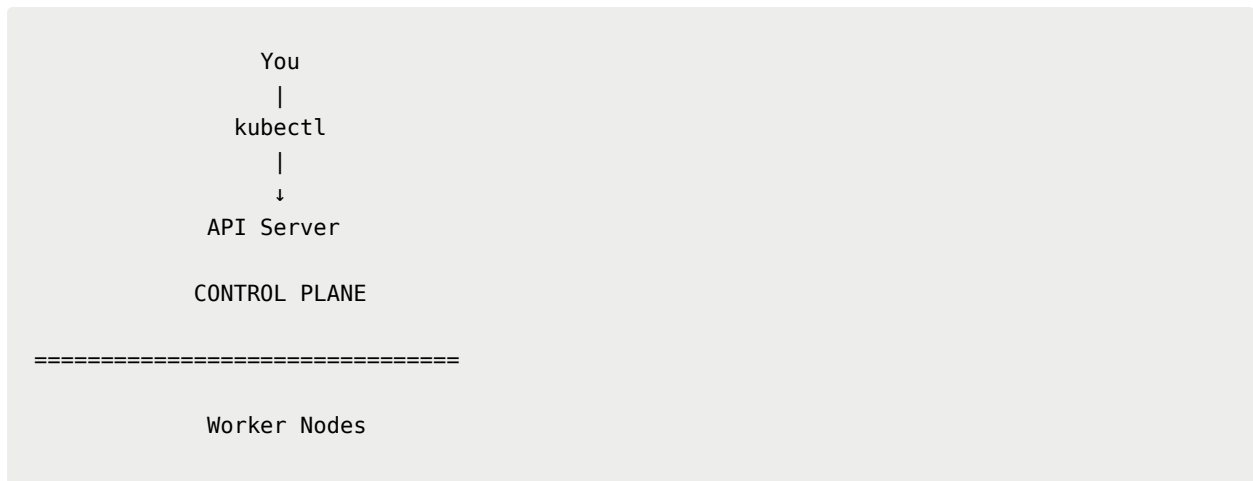
Conceptually means:

```
kubectl
  ↓
API Server,
give me information about the nodes.
```

The API Server processes the request and returns the response.

18. Architecture So Far

Our architecture currently looks like this:



The API Server gives us an entry point into Kubernetes.

But Kubernetes also needs to remember what we asked it to do.

19. Kubernetes Needs Memory — **etcd**

Suppose we tell Kubernetes:

```
I want 3 copies of my application.
```

If the Control Plane restarts tomorrow, Kubernetes should not forget this information.

It needs to remember things such as:

```
Which nodes exist?
```

```
Which applications should exist?
```

```
How many copies should exist?
```

```
What configuration exists?
```

What is currently running?

Kubernetes therefore needs persistent storage for its own cluster state.

That storage is:

etcd

etcd is a distributed key-value database used by Kubernetes to store cluster state.

A beginner-friendly mental model is:

etcd is Kubernetes' persistent memory.

Conceptually:

```
kubectl
  ↓
API Server
  ↓
etcd
```

A simplified imaginary entry could look like:

```
application = payment-service
desired-copies = 3
```

Important

etcd is **not** the database used by your application.

For example:

```
Spring Boot App
  ↓
MySQL
```

Your application does not store customer records in Kubernetes' **etcd**.

etcd stores Kubernetes' own cluster information.

20. kube-scheduler

Suppose we request Kubernetes to run an application.

There are multiple Worker Nodes available:

```
Worker A
CPU Available: 1 Core
RAM Available: 1 GB

Worker B
CPU Available: 8 Cores
RAM Available: 16 GB

Worker C
CPU Available: 2 Cores
RAM Available: 4 GB
```

Our application requires:

```
CPU: 2 Cores
RAM: 3 GB
```

Worker A cannot handle it.

Worker B can.

Worker C may also be suitable.

Someone has to select the appropriate worker.

That component is:

kube-scheduler

Its job is:

Find an appropriate Worker Node for a newly created workload.

Very Important

The Scheduler does **not** start the application.

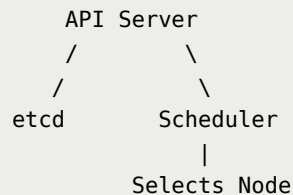
It only makes the placement decision.

For example:

Scheduler:

"This workload should run on Worker B."

So far:



21. kubelet — The Agent Running on Every Worker

The Scheduler may decide:

Run this workload on Worker Node B.

But something on Worker Node B still needs to make that decision happen.

That component is:

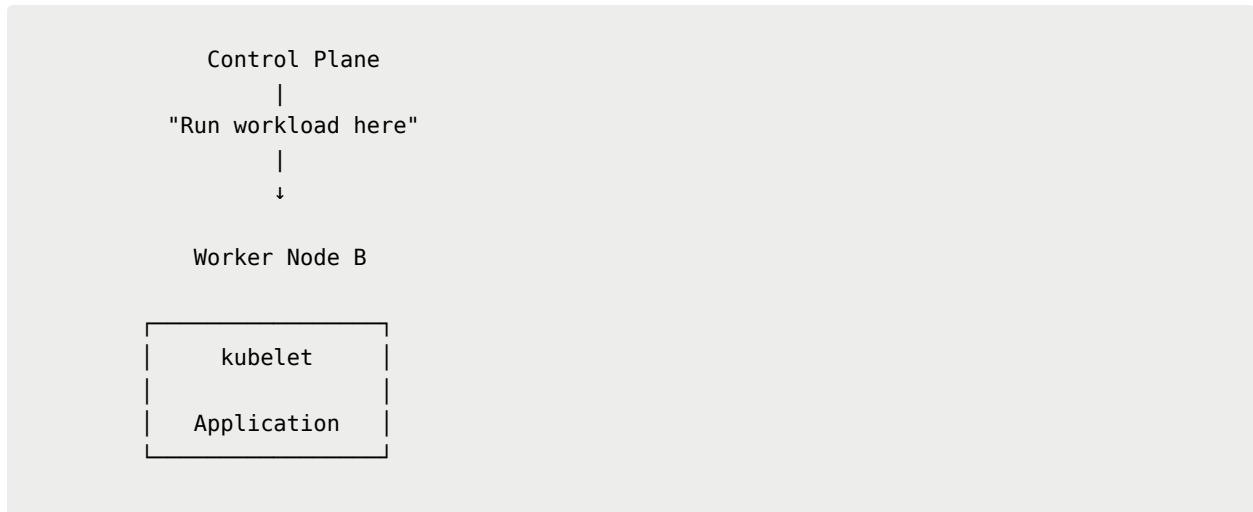
kubelet

Every Worker Node runs a **kubelet**.

Think of kubelet as:

Kubernetes' local agent running on a Node.

Conceptually:



Its responsibility is essentially:

Make sure the workloads assigned to this Node are actually running.

22. Container Runtime

Can the kubelet itself create Linux containers?

Not directly.

Containers are created and run through a **container runtime**.

Examples include:

```
containerd
CRI-O
```

The simplified flow is:

```
kubelet
  ↓
Container Runtime
```

↓
Container

So the kubelet supervises the work on the Node, while the container runtime actually handles container execution.

23. Controller Manager

We previously learned:

Desired State
vs
Actual State

Suppose we declare:

Desired Copies = 3

Initially:

Actual Copies = 3

Everything is correct.

Then one instance crashes:

Desired = 3
Actual = 2

Kubernetes needs something that constantly detects this mismatch.

This responsibility belongs to Kubernetes controllers, managed collectively through the:

Controller Manager

For beginners, think of controllers as:

Supervisors constantly checking whether the actual cluster matches the desired state.

Example:

```
Desired: 3
Actual: 2

Controller:
"We need one more."
```

The basic idea is:

```
Controller
  ↓
Observes Actual State
  ↓
Compares with Desired State
  ↓
Tries to remove the difference
```

This is the reconciliation model again.

24. Understanding the Control Plane

The major Control Plane components now make sense because each solves a specific problem.

API Server

Someone must receive Kubernetes requests.

```
API Server = Gateway
```

etcd

Someone must remember the cluster state.

```
etcd = Memory
```

Scheduler

Someone must decide where new workloads should run.

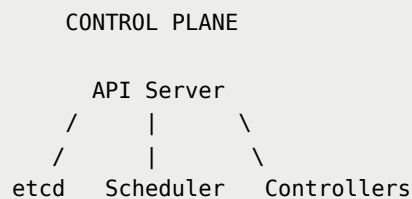
```
Scheduler = Placement Decision
```

Controller Manager

Someone must continuously check that the actual state matches the desired state.

```
Controllers = Supervisors
```

Together:



25. Worker Node Architecture

A Worker Node contains several important Kubernetes components:

```
Worker Node
```

```
|— kubelet
|— Container Runtime
|— kube-proxy
```

We already understand the first two.

kubelet

Ensures workloads assigned to this node are running.

Container Runtime

Actually creates and runs containers.

Now comes:

kube-proxy

Applications running inside Kubernetes need networking.

For example:

```
Frontend
  ↓
Backend
```

or:

```
User
  ↓
Application
```

Kubernetes has an abstraction called a **Service**, which we will study separately.

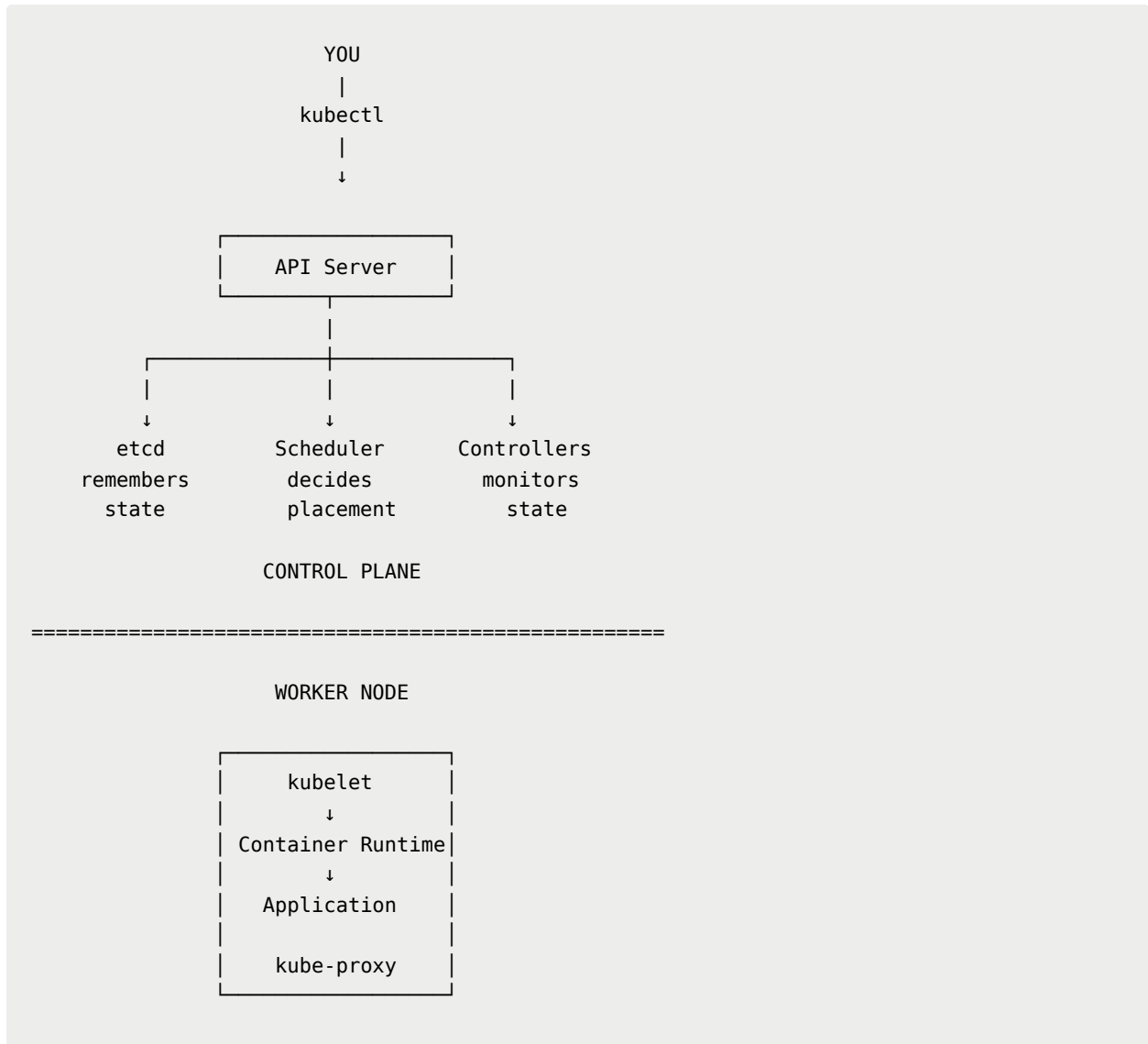
Historically, `kube-proxy` helps implement networking rules that direct Service traffic toward the correct Pods.

For now, remember:

```
kube-proxy
→ helps with Kubernetes Service networking
```

26. Complete Kubernetes Architecture

We can now combine everything:



A simple summary:

kubectl
→ Communicates with Kubernetes

API Server
→ Receives requests

etcd
→ Stores cluster state

Scheduler
→ Decides where workloads run

```
Controllers
→ Keep Desired State and Actual State aligned

kubelet
→ Agent running on every Node

Container Runtime
→ Actually runs containers

kube-proxy
→ Helps implement Service networking
```

27. Pods — From Absolute Zero

So far, our hierarchy looks like:

```
Kubernetes Cluster
      ↓
Control Plane
+
Worker Nodes
```

Now an important question appears:

| Does Kubernetes directly manage individual containers?

Normally, no.

Kubernetes introduces another abstraction:

Pod

28. Why Does Kubernetes Need Pods?

Imagine Kubernetes without Pods:

```
Worker Node
├─ Container A
```

```
|— Container B
|— Container C
```

The Scheduler could theoretically schedule individual containers:

```
Container A → Node 1
Container B → Node 2
```

But sometimes multiple containers are tightly connected.

For example:

```
Main Application Container
+
Helper Container
```

The helper container might:

- Collect logs
- Proxy network traffic
- Prepare files
- Perform some supporting task

These containers should remain together.

If Kubernetes scheduled them independently:

```
Main Application → Node 1

Helper Container → Node 8
```

the relationship would break.

We therefore need a single unit that represents:

┃ These containers belong together. Schedule and manage them as one unit.

That unit is a:

Pod

29. What Is a Pod?

A **Pod** is the smallest deployable unit Kubernetes manages.

A Pod contains:

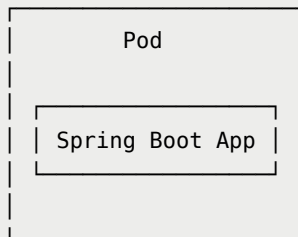
One or More Containers

Most commonly:

Pod
└─ One Application Container

Example:

Worker Node



The important distinction is:

Kubernetes doesn't normally say:

"Run Container X"

Instead:

"Run Pod X"

The Pod then contains the container.

30. Kubernetes Hierarchy

This hierarchy is extremely important:

```
Cluster
  ↓
Node
  ↓
Pod
  ↓
Container
```

Example:

```
Kubernetes Cluster
├── Node A
│   ├── Pod 1
│   │   └── Container
│   ├── Pod 2
│   │   └── Container
│   └── Pod 3
│       └── Container
└── Node B
    └── Pod 4
        └── Container
```

You will see this hierarchy repeatedly throughout Kubernetes.

31. Does Every Pod Have Multiple Containers?

No.

The normal structure is:

```
1 Pod
  ↓
1 Main Application Container
```

For a microservices architecture:

```
User-Service Pod
└─ User-Service Container

Payment-Service Pod
└─ Payment-Service Container

Order-Service Pod
└─ Order-Service Container
```

We normally do **not** put unrelated microservices inside the same Pod:

```
✗ One Pod
├─ User Service
├─ Payment Service
├─ Order Service
└─ Notification Service
```

These applications usually need to:

- Scale independently
- Fail independently
- Deploy independently

Therefore, they should normally live in separate Pods.

32. When Should a Pod Have Multiple Containers?

Multiple containers belong in the same Pod when they are **tightly coupled**.

Example:

```
Pod
├─ Main Application
└─ Helper Container
```

The key idea is:

```
They should be scheduled together.
```

If the Pod runs on Node B, all containers belonging to that Pod run together as part of that Pod.

33. What Do Containers Inside a Pod Share?

Containers inside the same Pod share certain resources.

Two important examples are:

1. Networking
2. Storage through shared volumes

34. Shared Network Identity

A Pod receives its own IP address.

For example:

```
Pod A
IP: 10.244.1.7
```

If the Pod contains two containers:

```
Pod A
IP: 10.244.1.7
```

```
|— App Container
|— Helper Container
```

both containers exist inside the same Pod networking context.

They can communicate with each other through:

```
localhost
```

For example, suppose the main application is running on:

```
Port 8080
```

The helper container inside the same Pod can access it using:

```
localhost:8080
```

because both containers share the Pod's network namespace.

35. Shared Storage

Containers inside the same Pod can also share Kubernetes volumes.

For example:



The main application could write:

```
/app/logs/app.log
```

and the helper container could read the same file through a shared volume.

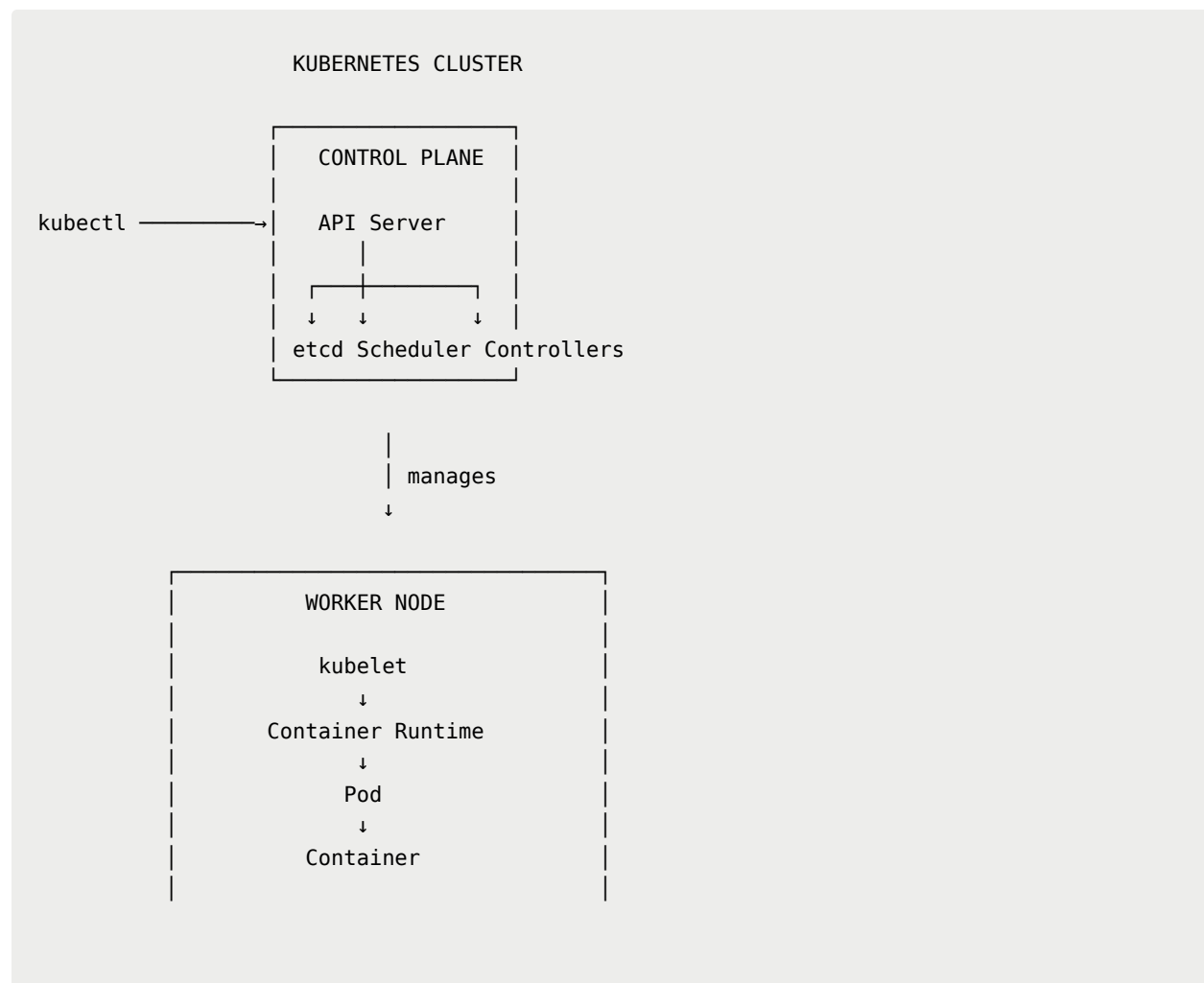
We will study Kubernetes Volumes separately.

For now, remember:

```
Same Pod
  ↓
Can share networking
  ↓
Can share volumes
```

36. Complete Beginner Mental Model

By this point, the full picture should look like this:



kube-proxy

37. Final Summary

Docker

Packages and runs containers

Kubernetes

Manages containerized applications at scale

Cluster

Collection of machines managed by Kubernetes

Node

A machine participating in the Kubernetes cluster

Control Plane

Brain of the cluster

Worker Node

Machine where application workloads run

kubectl

CLI used to communicate with Kubernetes

API Server

Gateway into Kubernetes

etcd

Persistent memory of Kubernetes cluster state

Scheduler

Chooses the Node on which a new workload should run

Controller Manager

Continuously tries to keep Actual State equal to Desired State

kubelet

Kubernetes agent running on every Node

Container Runtime

Actually creates and runs containers

kube-proxy

Helps with Service networking

Pod

Smallest deployable unit managed by Kubernetes

And the most important hierarchy:

```
Kubernetes Cluster
  ↓
Node
  ↓
Pod
  ↓
Container
```

Key Takeaway

The core idea behind Kubernetes is not simply:

| Run containers.

It is:

| **Declare how you want your application infrastructure to look, and let Kubernetes continuously work to keep reality in that state.**

That is the foundation on which Kubernetes concepts such as Deployments, ReplicaSets, Services, ConfigMaps, Volumes, autoscaling, and self-healing are built.